

STATS 507

Data Analysis in Python

Week4-1: More class method, Iterators, and Generators,
Exceptions

Recap: Class as programmer-defined types

Objects are instances of a class and are a data abstraction that captures:

- An **internal representation**
 - Through data attributes
- An **interface** for interacting with objects
 - Though methods (aka procedures/functions)
 - Defines behaviors but hides implementations

Creating the class (a parallel to function)

- Define the class **name**
- Define class data attributes
- Define procedural attributes

Using the class:

- Create new **instances** of the class
- Doing operations on the instances

Recap: creating our own class

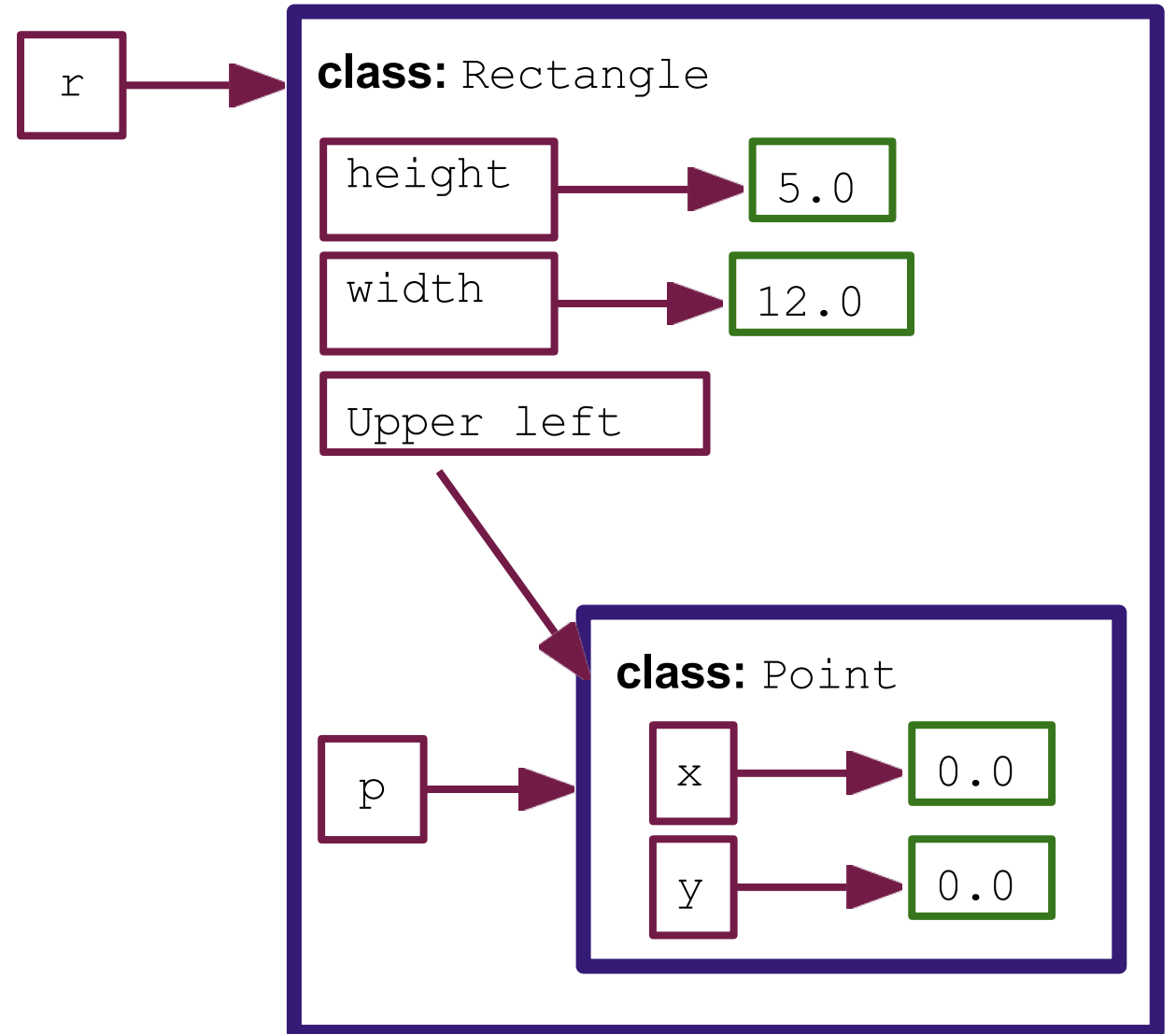
```
# YOUR CODE HERE
class Rectangle:
    def __init__(self, upper_left, height, width):
        if not isinstance(upper_left, Point):
            raise TypeError("upper_left must be a Point object")
        self.upper_left = upper_left
        self.height = height
        self.width = width

    def __str__(self):
        return f"Rectangle(upper_left={self.upper_left}, " \
            f"height={self.height}, width={self.width})"

    def area(self):
        return self.height * self.width

upper_left_corner = Point(0,0)
my_rectangle = Rectangle(upper_left_corner, 2, 4)
print(my_rectangle)
print(my_rectangle.area())

Rectangle(upper_left=Point(0, 0), height=2, width=4)
8
```



Recap: Inheritance

Inheritance is perhaps the most useful feature of object-oriented programming

Inheritance allows us to create new classes from old ones

Parent class
(base class/superclass)

Class definition Class name

Parent class

```
class Cat(Animal):  
    def speak(self):  
        print("meow")
```

Child class
(derived class/ subclass)

- **Inherit** all data and behaviors of the parent class
- **Add** more info(data)
- **Add** more behavior
- **Override** behavior

```
my_cat = Cat(7)  
my_cat.set_name("Fay")  
print(my_cat.get_name())  
print(my_cat.speak())
```

Fay
meow
None

One more thing for OOP on `__init__()`

`__init__()` for parent class

A special method to initialize some data attributes or perform initialize operations.

`self` represents an instance of a class, It is a parameter to refer to an instance of the class without creating one yet. **Always** going to be the first parameter of any method.

```
class Animal():
    def __init__(self, age):
        self.age = age
        self.name = None
    def __str__(self):
        return "animals: " + str(self.name) + ":" + str(self.age)
    def get_age(self):
        return self.age
    def get_name(self):
        return self.name
    def set_age(self, new_age):
        self.age = new_age
    def set_name(self, new_name = ''):
        self.name = new_name
```

`__init__()` for child class

```
class Cat(Animal):  
    def __init__(self, age, breed="Unknown"):  
        super().__init__(age)  
        self.breed = breed  
    def speak(self):  
        print("meow")  
    def get_breed(self):  
        return self.name  
    def set_breed(self, breed):  
        self.breed = breed
```

`super().__init__()` to ensure the attributes in parent class are properly initialized.

additional initialization steps specific to the subclass

```
my_cat = Cat(7)  
my_cat.set_name("Fay")  
my_cat.set_breed("British short hair")  
print(my_cat.get_name())  
print(my_cat.get_breed())  
print(my_cat.speak())
```

Fay
Fay
meow
None

1. Iterator and Generators

2. Exceptions and Assertions

Recall: iteration and iterable

Iterable: an object capable of returning its members one at a time:

- Lists are iterable.
- So are all **sequence** types (string, tuple...)
- Some non-sequence types like dict, file objects are also iterable.

We can loop over objects that are iterable.

```
nums = [1,2,3]
for n in nums:
    print(n)
```

1
2
3

```
s = 'abc'
for c in s:
    print(c)
```

a
b
c

How can we tell if an object is iterable?

The `__iter__()` method (another "dunder" method):

- An object can be looped over if it has the `__iter__()` method
- We say it is iterable.

```
nums = [1,2,3]
print(dir(nums))
```

```
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__',  
 '__eq__', '__format__', '__ge__', '__getattr__', '__getitem__', '__getstate__', '__gt__', '__hash__',  
 '__iadd__', '__imul__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__',  
 '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__se  
titem__', '__sizeof__', '__str__', '__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'i  
nsert', 'pop', 'remove', 'reverse', 'sort']
```

What is an iterator?

An iterator is an object that represents a “data stream”

Supports method `__next__()`:

returns next element of the stream/sequence

raises `StopIteration` error when there are no more elements left

Is list an iterator?

```
nums = [1,2,3]
print(next(nums))
```

```
-----
TypeError                                 Traceback (most recent call last)
Cell In[13], line 2
      1 nums = [1,2,3]
----> 2 print(next(nums))

TypeError: 'list' object is not an iterator
```

Getting an iterator

Lists are **not** iterators, so we first have to turn the list `t` into an iterator using the function `iter()`.

```
1 t = [1,2]
2 titer = iter(t)
3 next(titer)
```

1

Now, each time we call `next()`, we get the next element in the list. **Reminder:** `next(iter)` and `iter.__next__()` are equivalent.

```
1 next(titer)
```

2

Once we run out of elements, we get an error.

```
1 next(titer)
```

```
-----
StopIteration                                Traceback (most recent call last)
<ipython-input-20-105e88283d1e> in <module>()
----> 1 next(titer)
```

StopIteration:

Iterator in for loop

```
1 t = [1,2,3]
2 for x in t:
3     print(x)
4 print()
5 for x in iter(t):
6     print(x)
```

1
2
3

1
2
3

You are already familiar with iterators from previous lectures. When you ask Python to traverse an object `obj` with a for-loop, Python calls `iter(obj)` to obtain an iterator over the elements of `obj`.

These two for-loops are equivalent. The first one hides the call to `iter()` from you, whereas in the second, we are doing the work that Python would otherwise do for us by casting `t` to an iterator.

Creating an iterator – a dummy class

```
1 class dummy():
2     '''Class that is not iterable,
3     because it has neither __next__()
4     nor __iter__().'''
5
6 d = dummy()
7 for x in d:
8     print(x)
```

If we try to iterate over an object that is not iterable, we're going to get an error.

Objects of class `dummy` have neither `__iter__()` (i.e., doesn't support `iter()`) nor `__next__()`, so iteration is hopeless. When we try to iterate, Python is going to raise a `TypeError`.

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-30-fc084e213893> in <module>()
      5
      6 d = dummy()
----> 7 for x in d:
      8     print(x)

TypeError: 'dummy' object is not iterable
```

Creating our own iterator

`__next__()`: create our own `__next__()` method

```
1 class Squares():
2     '''Iterator over the squares.'''
3     def __init__(self):
4         self.n = 0
5     def __next__(self):
6         (self.n, k) = (self.n+1, self.n)
7         return(k*k)
8 s = Squares()
9 [next(s) for _ in range(10)]
```

`__next__()` is the important point, here.
It returns a value, the next square.

`next(iter)` is equivalent to calling
`__next__()`. Variable `_` in the list
comprehension is a placeholder, tells
Python to ignore the value.

```
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Can we loop over the `Squares`? Why?

Iterable v.s iterator

```
1 class Squares():
2     '''Iterator over the squares.'''
3     def __init__(self):
4         self.n = 0
5     def __next__(self):
6         (self.n, k) = (self.n+1, self.n)
7         return(k*k)
8 s = Squares()
9 for x in s:
10     print(x)
```

Merely being an iterator isn't enough, either!
for X in Y requires that object Y be iterable.

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-11-f0187d07bc4c> in <module>()
      7         return(k*k)
      8 s = Squares()
----> 9 for x in s:
     10     print(x)

TypeError: 'Squares' object is not iterable
```

Creating an iterable

```
1 class Squares():
2     '''Iterator over the squares.'''
3     def __init__(self):
4         self.n = 0
5     def __next__(self):
6         (self.n, k) = (self.n+1, self.n)
7         return(k*k)
8     def __iter__(self):
9         return(self)
10 s = Squares()
11 for x in s:
12     print(x)
```

Iterable means that an object has the `__iter__()` method, which returns an iterator. So `__iter__()` returns a new object that supports `__next__()`.

Now `Squares` supports `__iter__()` (it just returns itself!), so Python allows us to iterate over it.

0
1
4
9
16
25

This is an infinite loop.

Generator expressions

Recall that a list comprehension creates a list from an iterable

```
1 def square(k):  
2     return(k*k)  
3 [square(x) for x in range(17) if x%2==0]  
  
[0, 4, 16, 36, 64, 100, 144, 196, 256]
```

List comprehension computes and returns the whole list. What if the iterable were infinite? Then this list comprehension would never return!

```
1 s = Squares()  
2 [x**2 for x in s]
```

This list comprehension is going to be infinite! But I really ought to be able to **get an iterator** over the squares of the elements.

```
1 sqgen = (x**2 for x in s)  
2 sqgen
```

```
<generator object <genexpr> at 0x106d02780>
```

This is the motivation for **generator expressions**. Generator expressions are like list comprehensions, but they create an iterator rather than a list.

Generator expressions are written like list comprehensions, but with parentheses instead of square brackets.

Generators

Related to generator expressions are **generators**

Provide a simple way to write iterators (avoids having to create a new class)

```
1 def harmonic(n):  
2     return(sum([1/k for k in range(1,n+1)]))  
3 harmonic(10)
```

2.9289682539682538

Each time we call this function, a local namespace is created, we do a bunch of work there, and then all that work disappears when the namespace is destroyed.

```
1 def harmonic():  
2     (h,n) = (0,1)  
3     while True:  
4         (h,n) = (h+1/n, n+1)  
5         yield h  
6 h = harmonic()  
7 [next(h) for _ in range(3)]
```

[1.0, 1.5, 1.8333333333333333]

Alternatively, we can write `harmonic` as a **generator**. Generators work like functions, but they maintain internal state, and they `yield` instead of `return`. Each time a generator gets called, it runs until it encounters a `yield` statement or reaches the end of the `def` block.

Writing iterators using generators

```
1 def harmonic():
2     (h,n) = (0,1)
3     while True:
4         (h,n) = (h+1/n, n+1)
5         yield h
6 h = harmonic()
7 h
```

Python sees the `yield` keyword and determines that this should be a generator definition rather than a function definition.



```
<generator object harmonic at 0x1053b9fc0>
```

```
1 next(h)
```

```
1.0
```

```
1 next(h)
```

```
1.5
```

```
1 next(h)
```

```
1.8333333333333333
```

Writing iterators using generators

```
1 def harmonic():
2     (h,n) = (0,1)
3     while True:
4         (h,n) = (h+1/n, n+1)
5         yield h
6 h = harmonic()
7 h
```

Python sees the `yield` keyword and determines that this should be a generator definition rather than a function definition.

Create a new `harmonic` generator. Inside this object, Python keeps track of where in the `def` code we are. So far, no code has been run.

```
<generator object harmonic at 0x1053b9fc0>
```

```
1 next(h)
```

```
1.0
```

```
1 next(h)
```

```
1.5
```

```
1 next(h)
```

```
1.8333333333333333
```

Writing iterators using generators

```
1 def harmonic():
2     (h,n) = (0,1)
3     while True:
4         (h,n) = (h+1/n, n+1)
5         yield h
6 h = harmonic()
7 h
```

Python sees the `yield` keyword and determines that this should be a generator definition rather than a function definition.

```
<generator object harmonic at 0x1053b9fc0>
```

```
1 next(h)
```

```
1.0
```

Each time we call `next`, Python runs the code in `h` from where it left off until it encounters a `yield` statement.

```
1 next(h)
```

```
1.5
```

```
1 next(h)
```

```
1.8333333333333333
```

If/when we run out of `yield` statements (i.e., because we reach the end of the definition block), the generator returns a `StopIteration` error, as required of an iterator (not shown here).

In-class practice

1. Iterator and Generators

2. Exceptions and Assertions

Unexpected conditions

What happens when procedure execution hits an unexpected condition?

- Got an **exception**... to what was expected.

```
l = [1,2,3]
l[4]
```

```
-----
IndexError
Cell In[43], line 2
      1 l = [1,2,3]
----> 2 l[4]

IndexError: list index out of range
```

```
a
```

```
-----
NameError
Cell In[47], line 1
----> 1 a

NameError: name 'a' is not defined
```

```
int('exception')
```

```
-----
ValueError
Cell In[45], line 1
----> 1 int('exception')

ValueError: invalid literal for int()
```

```
"str"/2
```

```
-----
TypeError
Cell In[49], line 1
----> 1 "str"/2

TypeError: unsupported operand type(s)
```

Handling exceptions

Instead of crash, we can **handle** exceptions using:

```
try:                                if <all potentially problematic code succeeds>:
    # do some potentially          # great, all that code
    # problematic code            # just ran fine!
except:                              else:
    # do something to             # do something to
    # handle the problem          # handle the problem
```

Exception handler in Python

- If code in `try` block all succeed, `except` block will not be executed
- Exceptions raised by any statement in body of `try` are handled by the `except` statement

Example: without exception handling

Python will crash immediately

```
def divide_numbers(a, b):  
    result = a / b  
    return result  
print(divide_numbers(5,0))  
print("Done")
```

```
-----  
ZeroDivisionError                                Traceback (most recent call last)  
Cell In[73], line 4  
      2     result = a / b  
      3     return result  
----> 4 print(divide_numbers(5,0))  
      5 print("Done")  
  
Cell In[73], line 2, in divide_numbers(a, b)  
      1 def divide_numbers(a, b):  
----> 2     result = a / b  
      3     return result  
  
ZeroDivisionError: division by zero
```

Example: with exception handling

Python will be able to handle exception even though code in try block is problematic.

```
def divide_numbers(a, b):  
    try:  
        result = a / b  
        return result  
    except ZeroDivisionError:  
        return "Error: Division by zero is not allowed."  
print(divide_numbers(5,0))  
print("Done")
```

We do not have to specify the Error.

```
Error: Division by zero is not allowed.
```

```
Done
```

Example: can catch different errors

Can have separate except clauses to deal with a particular type of exception.

```
def divide_numbers(a, b):  
    try:  
        result = a / b  
        return result  
    except ZeroDivisionError:  
        return "Error: Division by zero is not allowed."  
    except TypeError:  
        return "Error: Please pass numeric values only."  
    except:  
        return "Something else went wrong..."  
  
divide_numbers(3, 'a string')
```

Only execute if this kind of error is raised in `try` block

Can associate other blocks with `try` block

Besides `except` blocks

- `else`
 - Body will always be executed **when** `try` block competes with no exceptions
- `finally`
 - Body will always be executed, regardless of whether an exception was raised or not
 - Useful for cleanup actions
 - Ex: close a file...

```
def divide_numbers(a, b):  
    try:  
        result = a / b  
    except ZeroDivisionError:  
        return "Error: Division by zero is not allowed."  
    else:  
        return f"The result is: {result}"  
    finally:  
        print("Execution complete.")
```

Execution complete.

'The result is: 2.0'

Assertions: a defensive programming tool

It relates to an assumption on the state of computation are as expected.

Use as assert statement to raise an `AssertionError` exception if assumptions are not met

```
assert <statement that should evaluate to be true>, "statement not true"
```

Assertion usage

To use as good defensive programming

- Check inputs to functions, but can be used anywhere
- Check outputs to functions to avoid propagating bad values
- Can make it easier to debug.

```
def divide(a, b):  
    assert b != 0, "Division by zero is not allowed"  
    return a / b  
  
# Test the function  
print(divide(10, 2)) # This will work fine  
print(divide(10, 0)) # This will raise an AssertionError
```

```
class Person:  
    def __init__(self, age):  
        assert age > 0, "Age must be positive"  
        self.age = age  
  
# Create a Person object  
p = Person(25) # Works fine  
p = Person(-5) # Raises AssertionError
```

Other things

HW3 out.

Coming next:

Algorithm thinking